Session-Based Authentication

a random ID in a cookie, the state on the server

Watch the 4-min video

In one line: after login the server stores the session (user ID, expiry) in a shared store and gives the browser only a long random ID in a cookie; protect that cookie with HttpOnly, Secure and SameSite, block CSRF, and enjoy instant revocation.

1 HTTP remembers nothing

You log in, close the tab, and return an hour later still logged in. HTTP is stateless, so something must carry your identity between requests. The classic answer is a **server-side session**: simple, battle-tested, and the default in Django, Rails, Laravel, Spring and ASP.NET. The cookie that carries it is what attackers go after.

2 How a session works

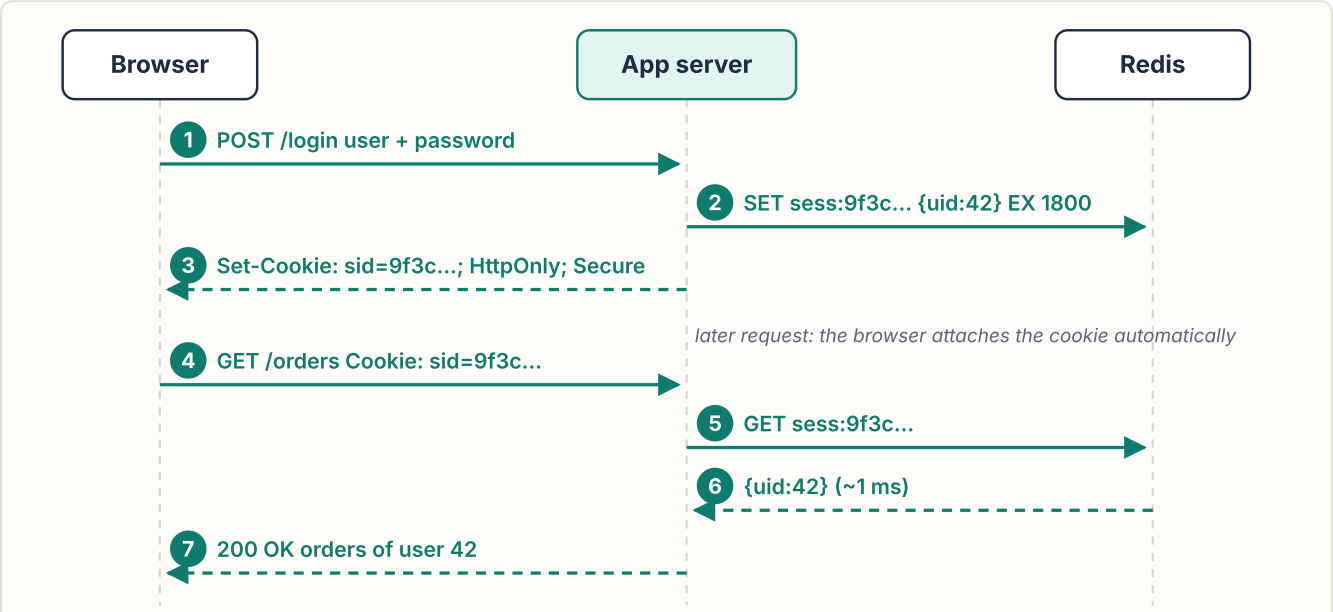


Figure 1. Login creates a session record in Redis and returns only its ID. Every later request carries the cookie, and the server looks the ID up.

The session ID must come from a cryptographically secure random generator and be long enough that guessing is hopeless: OWASP asks for at least **64 bits of entropy**; frameworks typically use 128 bits or more. The ID means nothing by itself; everything about the user stays on the server.

3 Where sessions live

In-process memory works for one server. Behind a load balancer, a user hitting a different server would look logged out, so either use sticky sessions (fragile: a server restart still logs users out) or, better, a **shared store** so every app server stays stateless. Redis is the usual choice: one key per session with a TTL, lookups around a millisecond. If Redis restarts with no persistence, every user is logged out, so run it with replication (and persistence if logouts would hurt).

4 Protect the cookie

```
Set-Cookie: __Host-sid=9f3c…; Path=/; HttpOnly; Secure; SameSite=Lax
```

- **HttpOnly**: JavaScript can't read the cookie, so an XSS bug can't steal the session ID (it can still act inside the page, so fix XSS too).
- **Secure**: sent only over HTTPS. The `__Host-` prefix additionally forces Secure, `Path=/` and no `Domain`, so subdomains can't overwrite it.
- **SameSite**: controls whether the cookie is sent on cross-site requests (table below).
- **Regenerate the ID at login** (and on privilege change) to stop **session fixation**, where an attacker plants a known ID before the victim logs in.

SameSite	Top-level link from another site (GET)	Cross-site POST, iframe, fetch	Use for
Strict	not sent	not sent	banking, admin consoles
Lax	sent	not sent	most session cookies; Chrome's default when unset
None (needs Secure)	sent	sent	embedded widgets, cross-site SSO; needs CSRF tokens

5 CSRF: the cookie rides along

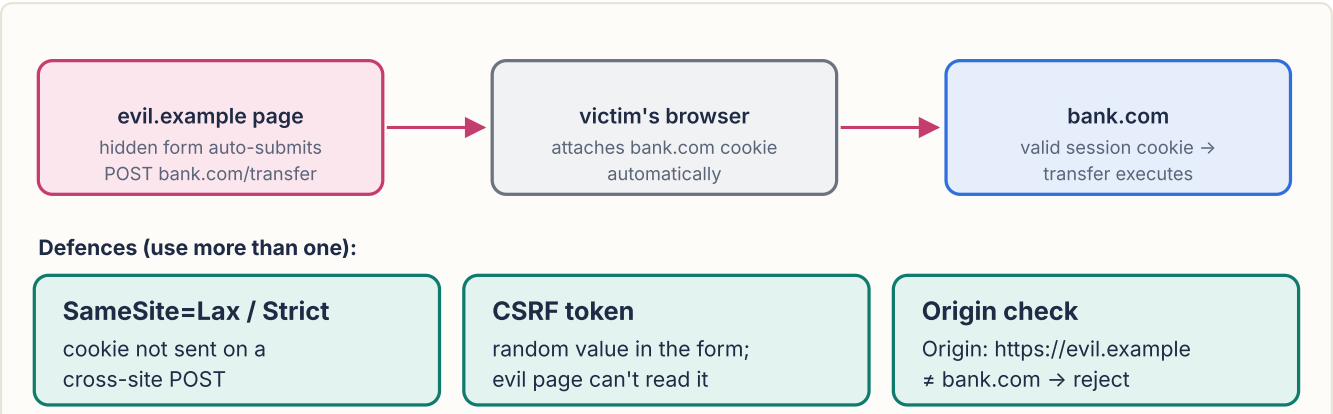


Figure 2. Cross-site request forgery: the attacker never sees the cookie; the browser sends it for them.

Because browsers attach cookies automatically, another site can make the victim's browser send an authenticated, state-changing request. Defend in layers: SameSite=Lax or Strict; a **CSRF token** (synchronizer token stored in the session and echoed in each form or header); and checking the `Origin` (or `Sec-Fetch-Site`) header on POST, PUT, PATCH and DELETE. Never change state on GET.

6 Revocation is the superpower

Logout deletes the record, and the cookie is useless at once. You can also log a user out of every device, expire idle sessions after, say, 30 minutes, and kill all sessions when the password changes. The price: a store lookup on every request, and the store must stay available. JWTs (lesson 41) make the opposite trade.

WORKED EXAMPLE: DJANGO SESSIONS IN REDIS

Point `CACHES` at a Redis cluster, then set `SESSION_ENGINE = "django.contrib.sessions.backends.cache"`, `SESSION_COOKIE_SECURE = True`, `SESSION_COOKIE_HTTPONLY = True`, `SESSION_COOKIE_SAMESITE = "Lax"`, and keep `CsrfViewMiddleware` on. Django's `login()` already rotates the session key. Any app server can serve any user, and logout works everywhere instantly.

COMMON MISTAKES

- Storing sessions in local memory behind a load balancer, so users get random logouts.
- Keeping the same session ID before and after login (session fixation).
- Setting `SameSite=None` for convenience and having no CSRF token.
- Logout that only deletes the cookie in the browser while the server-side session stays valid.

7 Key terms

Session ID: random, unguessable key to server-side state

HttpOnly: cookie hidden from JavaScript

SameSite: limits sending the cookie on cross-site requests

Session fixation: attacker plants a known ID before login

CSRF: another site triggers an authenticated request

Idle / absolute timeout: expiry after inactivity / after a fixed lifetime

8 Interview questions

Q1. How do you protect session cookies? BASIC

Use a long random ID with HttpOnly, Secure and SameSite=Lax or Strict, regenerate it at login, and expire and revoke sessions on the server.

Q2. What is CSRF? BASIC

Cross-site request forgery: another site makes the victim's browser send an authenticated request, because cookies are attached automatically. Block it with SameSite cookies, CSRF tokens and Origin checks on state-changing requests.

Q3. Users get logged out randomly after you scale from one server to four. Why? SCENARIO

Sessions live in each server's memory, so a request routed to another server finds no session. Move them to a shared store like Redis (or, as a stopgap, sticky sessions).

Q4. A user reports a stolen laptop. What do you do with sessions? SCENARIO

Delete all of that user's sessions in the store (track session IDs per user), which logs every device out at once, then force a password change.

Q5. HttpOnly is set. Is XSS now harmless? TRICKY

No. Script can't read the cookie, but it runs in the page, so it can still send requests with the session and read the responses. Fix XSS itself.

Q6. Does SameSite=Lax make CSRF tokens unnecessary? TRICKY

Not completely: Lax still sends the cookie on top-level GETs (so GETs must never change state), sibling subdomains count as same-site, and old browsers ignore it. Keep tokens or Origin checks too.

Next: SD-M-003 · JWT Authentication backendroadmap.com